

Informe de Laboratorio 08

Tema: Django REST Framework

Nota

Estudiantes	Escuela	Asignatura
Aragón Carpio Fredy José Arce Coaquira Diego Marcelo Bravo Rojas José Manuel Carrillo Villalta Gustavo Alonso	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: III Código: 2502117 Grupo: Desarrolladores

Laboratorio	Tema	Duración
08	Django REST Framework	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	25 de Mayo de 2026	01 de Junio de 2026

Índice

1. Laboratorio 08: Django REST Framework	3
1.1. Descripción y Objetivos	3
1.2. Entregables	3
1.3. Entorno de Implementación	3
2. Equipos, materiales y temas utilizados	4
3. URL del Repositorio en GitHub	4
4. Desarrollo de actividades	4
4.1. Estructura del proyecto	4
4.2. Instalación y configuración de DRF	5
4.3. Serializadores	6
4.4. Vistas JSON	7
4.5. JSON anidados	8
4.5.1. Ejemplo de respuesta JSON anidada	8

4.6. Registro de EndPoints	9
4.6.1. Tabla de endpoints disponibles	9
4.7. Panel de la API DRF	10
4.7.1. Vista principal del panel DRF	10
4.7.2. Detalle de un endpoint	11
4.8. Pruebas con cliente REST	11
4.8.1. GET Listado de registros	12
4.8.2. POST Creación de registro	13
4.8.3. PUT Actualización de registro	14
4.8.4. DELETE Eliminación de registro	15
5. Resultados	15
6. Autoevaluación	16
7. Rúbrica de Calificación	17
8. Referencias	17

1. Laboratorio 08: Django REST Framework

Django REST Framework (DRF) es una librería de alto nivel que facilita la construcción de APIs RESTful sobre proyectos Django. En este laboratorio se implementaron serializadores, vistas basadas en `APIView` y `ViewSet`, endpoints registrados en `urls.py`, y se habilitaron las operaciones GET, POST, PUT y DELETE de forma anónima, sin uso de JWT. También se construyeron serializadores anidados para exponer consultas complejas en formato JSON.

La práctica estuvo orientada a integrar DRF sobre el proyecto existente, configurarlo correctamente en `settings.py`, documentar cada endpoint con capturas del panel de la API y de clientes REST como Postman o SoapUI, y desplegar el proyecto en Render con base de datos en Supabase.

1.1. Descripción y Objetivos

Esta práctica reforzó el uso de Django REST Framework como herramienta para exponer datos del modelo relacional en formato JSON a través de una API REST. Los objetivos principales fueron:

- Instalar y configurar Django REST Framework en el proyecto Django existente.
- Crear serializadores a partir de los modelos del sistema.
- Implementar vistas JSON que expongan datos mediante la API de DRF.
- Construir serializadores y vistas anidadas para consultas complejas.
- Registrar los endpoints en `urls.py` para su consumo externo.
- Permitir las operaciones GET, POST, PUT y DELETE de forma anónima (sin JWT).
- Capturar evidencias del panel de la API DRF y de los requests con un cliente REST.
- Desplegar la aplicación en Render y configurar la base de datos en Supabase.
- Elaborar el `README.md` y el informe correspondiente.

1.2. Entregables

Los entregables del laboratorio se organizan en el repositorio, el video de demostración, el despliegue en Render y el informe final.

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>
- **Video:** <https://youtu.be/COMPLETAR>
- **Informe PDF:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab8-DAW.pdf>
- **Deploy en Render:** <https://cafesys-backend.onrender.com>
- **Base de datos Supabase:** <https://supabase.com/dashboard/project/uybtqkpxkcivbzpgcsm>

1.3. Entorno de Implementación

Para el desarrollo se utilizó Python con Django y Django REST Framework, un entorno virtual para la instalación aislada de dependencias, editor VS Code, Docker, Docker Compose, y PostgreSQL como motor de base de datos. El despliegue en producción se realizó en Render, conectando con la base de datos alojada en Supabase.

- Python 3.10 o superior.
- Django.
- Django REST Framework.
- Entorno virtual `venv`.
- Docker y Docker Compose.

- PostgreSQL 17.
- Visual Studio Code.
- Git y GitHub.
- Render (despliegue).
- Supabase (base de datos en la nube).

2. Equipos, materiales y temas utilizados

- Sistema Operativo: Windows 11.
- Python y Django.
- Django REST Framework.
- Entorno virtual `venv`.
- Docker y Docker Compose.
- Visual Studio Code.
- PostgreSQL.
- Git y GitHub.
- Postman o SoapUI (cliente REST para pruebas).
- Render (plataforma de despliegue).
- Supabase (base de datos PostgreSQL en la nube).
- Navegador web para el panel de la API DRF.

3. URL del Repositorio en GitHub

El proyecto desarrollado en este laboratorio se encuentra documentado en GitHub junto con el informe PDF y el `README.md`.

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>
- **Informe PDF:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab8-DAW.pdf>

4. Desarrollo de actividades

4.1. Estructura del proyecto

La estructura del proyecto Django con DRF mantiene la organización estándar del laboratorio anterior, añadiendo los archivos `serializers.py` y la configuración actualizada de `urls.py` para los endpoints de la API.

```
cafesys-project/  
|  
+-- backend/                                <- Entorno del Servidor (Django)  
| |  
| +-- apps/  
|
```

```
| | +-- core/                <- Aplicacion Principal del Sistema
| | +-- migrations/         <- Historial de Migraciones a Supabase
| | +-- models/             <- Definicion de Tablas (Ej. Product)
| | +-- admin.py            <- Registro en Panel de Administracion
| | +-- apps.py             <- Configuracion de la App
| | +-- serializers.py      <- Conversion de Modelos a JSON
| | +-- urls.py             <- Enrutamiento de la API
| | +-- views.py            <- Logica de Controladores/Endpoints
| |
| +-- cafesys/              <- Configuracion Global de Django
| | +-- settings.py         <- Ajustes Generales
| | +-- urls.py             <- Enrutador Raiz del Proyecto
| | +-- asgi.py             <-
| | +-- wsgi.py             <-
| |
| +-- venv/                 <- Entorno Virtual Local (Python)
| +-- .env                  <- Variables de Entorno Secretas
| +-- .gitignore
| +-- Dockerfile            <- Configuracion del Contenedor Backend
| +-- manage.py             <- Gestor de Comandos de Django
| +-- README.md
| +-- requirements.txt       <- Dependencias (django-cors-headers, etc.)
|
+-- bd/                     <- Base de Datos y Modelado
| +-- cafesysdb.sql         <- Script de Respaldo SQL
| +-- DER-dgdiagram.io.png  <- Diagrama Entidad-Relacion (Web)
| +-- DER-Resumen.png       <- Imagen de Resumen del Modelo BD
|
+-- frontend/               <- Entorno de Cliente (Angular v21)
| +-- .gitignore
|
+-- informes/               <- Documentacion Academica UNSA
| +-- ...                   <- Archivos LaTeX / Informes de Lab
|
+-- docker-compose.yml      <- Orquestador de Contenedores Locales
+-- render.yaml             <- Configuracion para Deploy en Render
+-- README.md               <- Documentacion General del Repositorio
```

4.2. Instalación y configuración de DRF

Django REST Framework se instala mediante `pip` y se registra en `INSTALLED_APPS` del archivo `settings.py`. Adicionalmente se configuraron los permisos para permitir acceso anónimo a todos los endpoints, sin requerir autenticación JWT.

Listing 1: Instalación de Django REST Framework.

```
pip install djangorestframework
pip freeze > requirements.txt
```

Listing 2: Registro de DRF en `settings.py`.

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
```

```
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'rest_framework',
'apps.core',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.AllowAny',
    ],
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
        'rest_framework.renderers.BrowsableAPIRenderer',
    ],
}
```

4.3. Serializadores

Los serializadores traducen los objetos del modelo Django a formato JSON y viceversa. Se crearon serializadores individuales para cada modelo, utilizando `ModelSerializer` como clase base, lo que permite generar automáticamente los campos a partir del modelo.

Listing 3: Ejemplo de serializadores en `serializers.py`.

```
from rest_framework import serializers
from .models import Users, Products, Orders, OrderDetails

# 1. USUARIOS (Control de Clientes y Personal)
class UsersSerializer(serializers.ModelSerializer):
    class Meta:
        model = Users
        fields = ('id', 'firstName', 'lastName', 'email', 'role', 'status')

# 2. PRODUCTOS (El Catálogo de la Cafetería)
class ProductsSerializer(serializers.ModelSerializer):
    category_name = serializers.CharField(source='category.name', read_only=True)

    class Meta:
        model = Products
        fields = ('id', 'name', 'description', 'price', 'imageUrl', 'category', 'category_name', 'status')

# 3. PEDIDOS Y DETALLES (El Carrito de Compras)
class OrderDetailsSerializer(serializers.ModelSerializer):
    product_name = serializers.CharField(source='product.name', read_only=True)

    class Meta:
        model = OrderDetails
        fields = ('id', 'product', 'product_name', 'quantity', 'unitPrice', 'subtotal')
```

```
class OrdersSerializer(serializers.ModelSerializer):
    details = OrderDetailsSerializer(many=True, read_only=True)

    class Meta:
        model = Orders
        fields = ('id', 'user', 'orderStatus', 'total', 'notes', 'details', 'created')
```

4.4. Vistas JSON

Las vistas consumen los serializadores y exponen los datos del modelo en formato JSON. Se implementaron utilizando APIView o ModelViewSet, habilitando los métodos HTTP necesarios para las operaciones CRUD.

Listing 4: Vistas JSON en views.py.

```
from rest_framework import viewsets, filters
from .models import Users, Products, Orders
from .serializers import UsersSerializer, UsersWriteSerializer, ProductsSerializer,
    OrdersSerializer

# 1. VIEWS DE USUARIOS (Con lógica dinámica para proteger contraseñas)
class UsersViewSet(viewsets.ModelViewSet):
    queryset = Users.objects.all()
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['firstName', 'lastName', 'email']
    ordering_fields = ['lastName', 'email']
    ordering = ['lastName']

    def get_serializer_class(self):
        # Si se va a registrar o editar un usuario, usa el Serializer de escritura con
        # passwordHash
        if self.action in ('create', 'update', 'partial_update'):
            return UsersWriteSerializer
        # Para listar o ver el perfil, usa el Serializer limpio
        return UsersSerializer

# 2. VIEWS DE PRODUCTOS (Optimizado para evitar el problema de consultas N+1)
class ProductsViewSet(viewsets.ModelViewSet):
    # select_related precarga los datos de las categorías en una sola consulta SQL
    queryset = Products.objects.select_related('category').all()
    serializer_class = ProductsSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['name', 'description', 'category__name']
    ordering_fields = ['name', 'price']
    ordering = ['name']

# 3. VIEWS DE PEDIDOS (Optimizado para precargar carritos de compra anidados)
class OrdersViewSet(viewsets.ModelViewSet):
    # prefetch_related carga eficientemente los detalles OneToMany del pedido
    queryset = Orders.objects.select_related('user').prefetch_related('details',
        'details__product').all()
    serializer_class = OrdersSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
```

```
search_fields = ['user__firstName', 'user__lastName', 'id']
ordering_fields = ['created', 'total']
ordering = ['-created']
```

4.5. JSON anidados

Para consultas complejas que involucran relaciones de uno a muchos (One-to-Many), se construyeron estructuras anidadas. Un ejemplo de esto es la relación entre un pedido (**Orders**) y su desglose de productos (**OrderDetails**), donde el serializador principal incluye el conjunto de detalles del carrito de compras dentro de una misma respuesta unificada.

Listing 5: Serializadores anidados para el flujo de pedidos.

```
from rest_framework import serializers
from .models import Orders, OrderDetails

class OrderDetailsSerializer(serializers.ModelSerializer):
    product_name = serializers.CharField(source='product.name', read_only=True)

    class Meta:
        model = OrderDetails
        fields = ('id', 'product', 'product_name', 'quantity', 'unitPrice', 'subtotal')

class OrdersSerializer(serializers.ModelSerializer):
    # Relacion inversa mapeada mediante el parametro many=True
    details = OrderDetailsSerializer(many=True, read_only=True)

    class Meta:
        model = Orders
        fields = ('id', 'user', 'orderStatus', 'total', 'notes', 'details', 'created')
```

4.5.1. Ejemplo de respuesta JSON anidada

Al realizar una petición HTTP GET al endpoint de un pedido específico, el backend de Django expone los datos estructurados en formato JSON de la siguiente manera:

Listing 6: Estructura de la respuesta JSON para un pedido anidado.

```
{
  "id": 104,
  "user": 12,
  "orderStatus": "pending",
  "total": "24.50",
  "notes": "Entregar en la entrada de la Facultad de Sistemas.",
  "created": "2026-05-31T15:20:00Z",
  "details": [
    {
      "id": 215,
      "product": 3,
      "product_name": "Cafe Americano Grande",
      "quantity": 2,
      "unitPrice": "7.00",
      "subtotal": "14.00"
    },
    {
      "id": 216,
```



```
        "product": 7,  
        "product_name": "Sandwich de Polto con Queso",  
        "quantity": 1,  
        "unitPrice": "10.50",  
        "subtotal": "10.50"  
    }  
]  
}
```

4.6. Registro de EndPoints

Los endpoints de la API fueron registrados en `urls.py` utilizando el Router de DRF o rutas manuales con `path()`, según el tipo de vista implementada.

Listing 7: Registro de endpoints principales en `urls.py`.

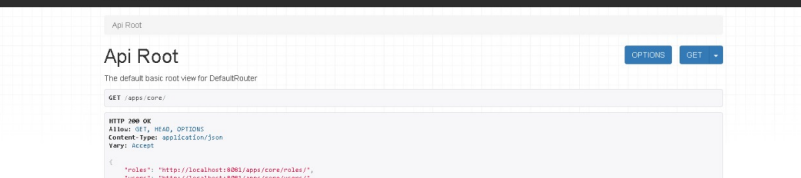
```
from django.urls import path, include  
from rest_framework.routers import DefaultRouter  
from .views import UsersViewSet, ProductsViewSet, OrdersViewSet  
  
# Creacion del enrutador mecanizado de DRF  
router = DefaultRouter()  
  
# Registro exclusivo de las tres entidades core del negocio  
router.register(r'users', UsersViewSet, basename='user')  
router.register(r'products', ProductsViewSet, basename='product')  
router.register(r'orders', OrdersViewSet, basename='order')  
  
# Exportacion de las URLs dinámicas hacia la app global  
urlpatterns = [  
    path('', include(router.urls)),  
]
```

4.6.1. Tabla de endpoints disponibles

A continuacion se detallan las rutas dinamicas generadas automaticamente por el sistema de enrutamiento para el consumo de la aplicacion frontend:

Tabla 1: Mapeo final de endpoints REST expuestos por el modulo core del backend.

Django REST Framework incluye una interfaz web navegable que permite explorar y probar los endpoints directamente desde el navegador. A continuación se muestran las capturas del panel de la API.



localhost:8081/apps/core/

Django REST framework

Api Root

Api Root

The default basic root view for DefaultRouter

OPTIONS GET

GET /apps/core/

```

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
  "rules": [
    "http://localhost:8081/apps/core/rules/",
    "users": "http://localhost:8081/apps/core/users/",
    "categories": "http://localhost:8081/apps/core/categories/",
    "products": "http://localhost:8081/apps/core/products/",
    "inventory": "http://localhost:8081/apps/core/inventory/",
    "orders": "http://localhost:8081/apps/core/orders/",
    "products-primitive": "http://localhost:8081/apps/core/products-primitive/",
    "local": "http://localhost:8081/apps/core/local/",
    "orders": "http://localhost:8081/apps/core/orders/",
    "order-details": "http://localhost:8081/apps/core/order-details/",
    "cars": "http://localhost:8081/apps/core/cars/",
    "vehicles": "http://localhost:8081/apps/core/vehicles/",
    "vehicles": "http://localhost:8081/apps/core/vehicles/",
    "reviews": "http://localhost:8081/apps/core/reviews/",
    "reviews": "http://localhost:8081/apps/core/reviews/"
  ]
}
```

Página 10

4.7.2. Detalle de un endpoint

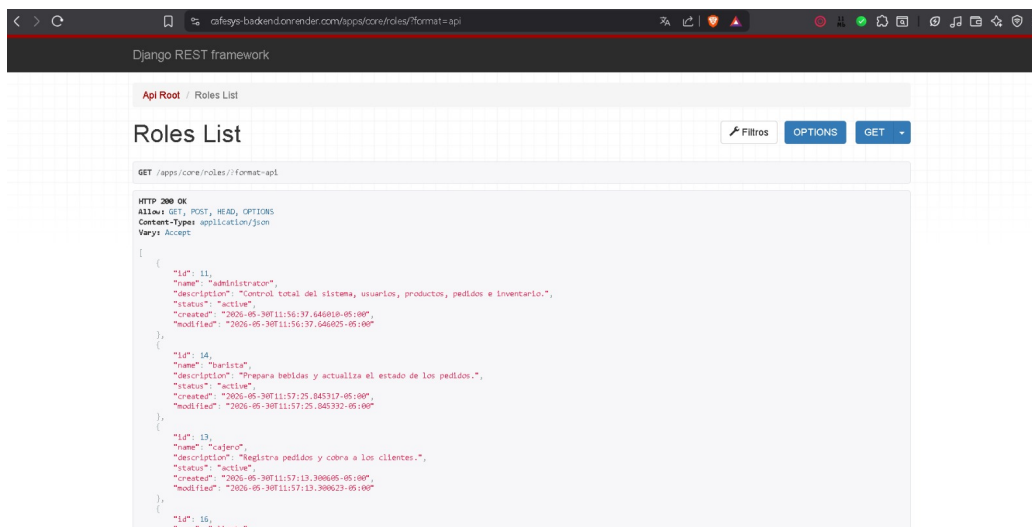


Figura 2: Vista de detalle de un endpoint en el panel de DRF.

4.8. Pruebas con cliente REST

Las operaciones CRUD fueron probadas utilizando Postman (o SoapUI) como cliente REST. A continuación se documentan las capturas de cada tipo de request.

4.8.1. GET Listado de registros

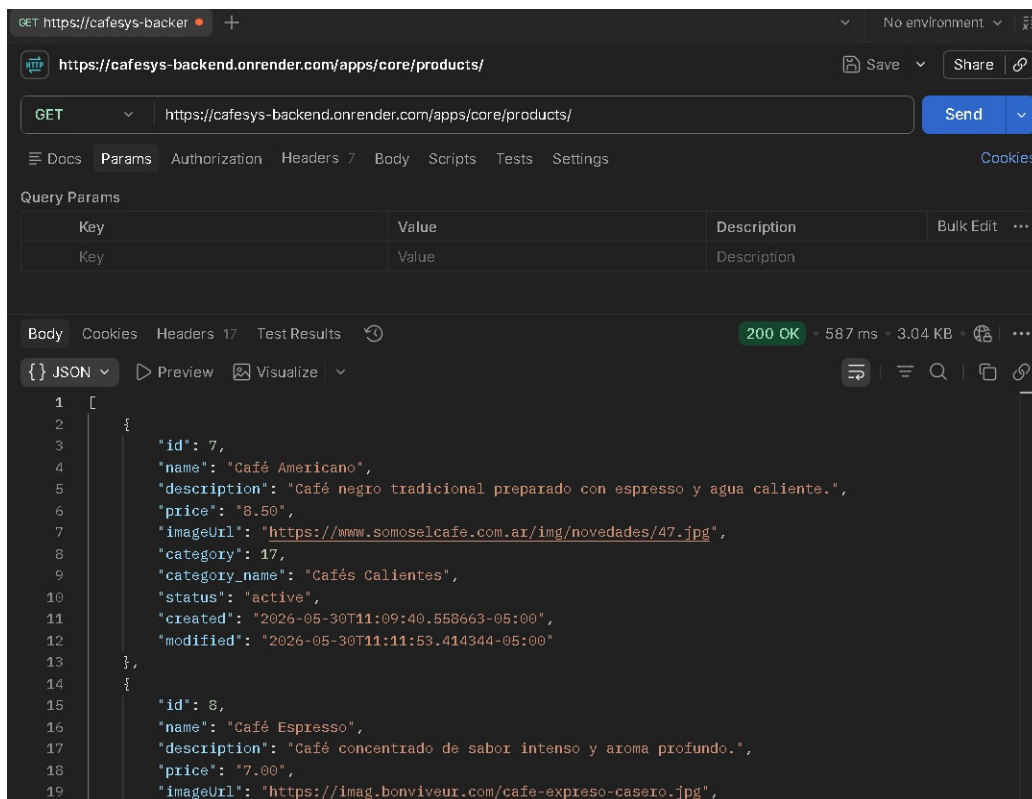


Figura 3: Request GET: listado de registros desde Postman.

4.8.2. POST Creación de registro

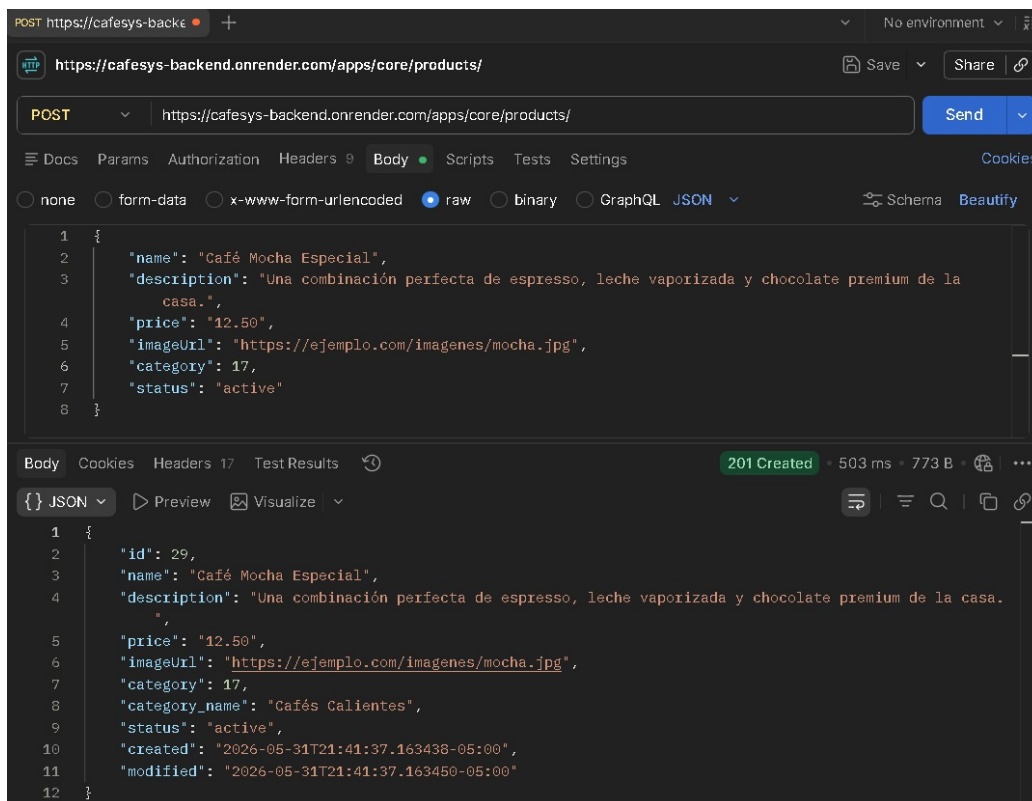


Figura 4: Request POST: creación de un nuevo registro.

4.8.3. PUT Actualización de registro

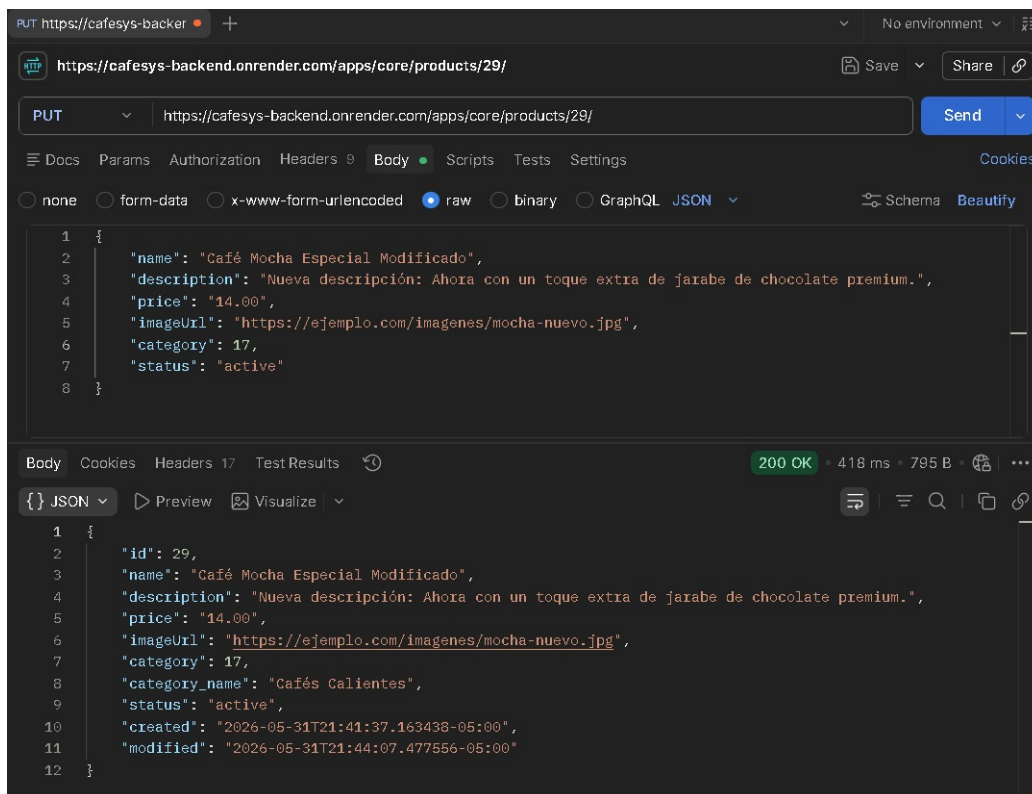


Figura 5: Request PUT: actualización de un registro existente.

4.8.4. DELETE Eliminación de registro

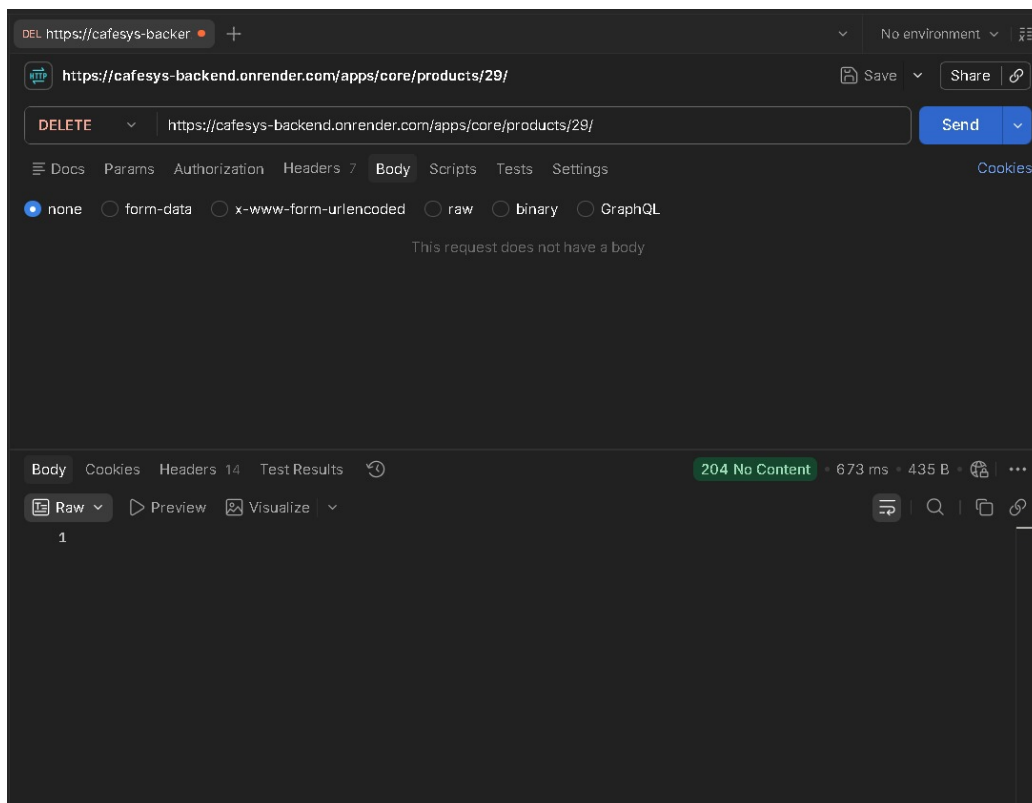


Figura 6: Request DELETE: eliminación de un registro.

5. Resultados

Los resultados evidencian que los serializadores fueron creados correctamente, los endpoints quedaron registrados y funcionando, y las operaciones GET, POST, PUT y DELETE operan de forma anónima. El despliegue en Render permitió acceder a la API desde una URL pública conectada a la base de datos en Supabase.

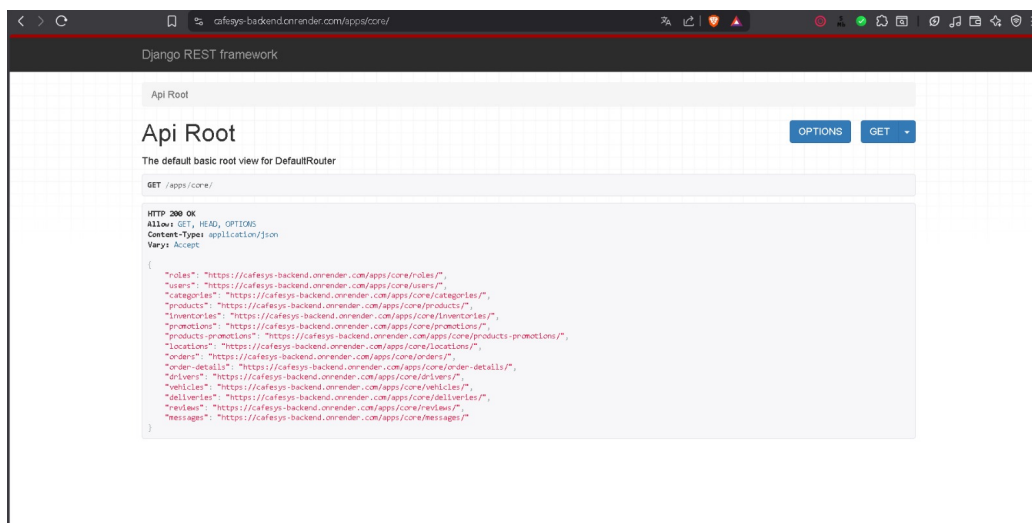


Figura 7: Vista general de los endpoints disponibles en la API desplegada.

6. Autoevaluación

Nombre	Rol	Porcentaje
Aragón Carpio Fredy José	Creación de serializadores	100 %
Arce Coaquira Diego Marcelo	Creación de vistas JSON y endpoints	100 %
Bravo Rojas José Manuel	Elaboración del informe	100 %
Carrillo Villalta Gustavo Alonso	Despliegue en Render/Supabase	100 %
Total		100 %

Tabla 2: Autoevaluación del laboratorio.

7. Rúbrica de Calificación

Ítem	Descripción	Puntaje
Entorno Virtual	Uso correcto del entorno virtual en proyectos de Python.	2
Instalación y configuración de DRF	Realiza detalladamente los pasos para instalar y configurar Django REST Framework.	2
Serializadores	Explica la creación de Serializadores.	4
Vistas JSON	Explica la creación de vistas que producen JSON.	4
JSON anidados	Crea Serializadores y Vistas para producir JSON anidados para consultas complejas.	4
Informe	El laboratorio tiene un informe que detalla todos los pasos necesarios para el desarrollo de la práctica.	4
Prueba*	Se tomaron en cuenta todas las consideraciones y recomendaciones, evidenciando trabajo en equipo.	-0
Total		20

Tabla 3: Rúbrica de evaluación del laboratorio. *La autoevaluación es obligatoria.

8. Referencias

- <https://www.django-rest-framework.org/>
- <https://www.django-rest-framework.org/api-guide/permissions/>
- <https://www.geeksforgeeks.org/python/adding-permission-in-api-django-rest-framework/>
- <https://blog.enriqueoriol.com/2015/03/django-rest-framework-serializers.html>
- <https://www.geeksforgeeks.org/python/serializers-django-rest-framework/>